

ARQUITETURAS DE SOFTWARE

Resumo Completo para Estudo + Simulado com Gabarito

Prof. Esp. Leonardo Dias | UniFOA | Curso de Sistemas de Informacao

PARTE 1 — RESUMO PARA ESTUDO

Aula 01 — Introducao e Atributos Essenciais

A arquitetura de software e a estrutura fundamental de um sistema: define componentes, relacionamentos e principios que guiam seu projeto e evolucao. Responde a pergunta: como o sistema sera organizado para atender requisitos funcionais e nao funcionais?

Atributos Essenciais de Qualidade

Conceito	Descrição
Modularidade	Divisao do sistema em partes coesas e fracamente acopladas. Favorece reuso, testabilidade e tolerancia a mudancas. Ex: modulo de autenticacao independente do modulo de relatorios.
Escalabilidade	Capacidade de crescer sem perder desempenho. Vertical: mais CPU/memoria na mesma maquina. Horizontal: adicionar novas instancias via Docker/Kubernetes.
Manutenibilidade	Facilidade para modificar o sistema com seguranca. Depende de Clean Code, testes automatizados e separacao de responsabilidades.
Seguranca	Protecao contra acessos nao autorizados ou falhas criticas no sistema.
Confiabilidade	Disponibilidade continua e resistencia a falhas em producao.
Performance	Rapidez na resposta das operacoes do sistema.

Niveis Hierarquicos de Abstracao

Arquitetura vs Design vs Implementacao

Conceito	Descrição
Arquitetura	Alto nivel. Define grandes blocos e interacoes. Ex: API REST .NET + banco SQL Server + front-end Angular.
Design	Nivel intermediario. Organiza internamente modulos e classes. Ex: padrao Repository + MVC no back-end.
Implementacao	Nivel mais baixo. Codigo fonte, testes e scripts. Ex: metodo GetCustomers() no repositorio.

Aula 02 — Estilos e Padroes Arquiteturais

Estilos arquiteturais sao formas gerais de organizar um sistema. Cada um traz vantagens, desvantagens e contextos de uso distintos.

Principais Estilos Arquiteturais

Conceito	Descrição
----------	-----------

Camadas (Layered)	Sistema organizado em níveis hierárquicos: UI, lógica de negócio e dados. Comum em sistemas bancários tradicionais.
Cliente-Servidor	Clientes solicitam serviços, servidores respondem. Ex: aplicação desktop acessando banco de dados remoto.
SOA	Serviços independentes com interfaces bem definidas. Precursor dos microsserviços. Comum em empresas que expõem APIs de pagamento.
Microsserviços	Cada funcionalidade é um serviço independente comunicado via APIs. Usado por Netflix, Amazon.
Event-Driven	Componentes reagem a eventos de outros. Ex: e-mail disparado automaticamente após compra finalizada.
Pipeline/Batch	Processamento em etapas sequenciais. Ex: compiladores (análise léxica > sintática > código objeto).

Principais Padrões Arquiteturais

Conceito	Descrição
MVC	Model-View-Controller: separa dados, lógica de negócio e interface gráfica.
MVVM	Model-View-ViewModel: usado em apps desktop/mobile como Angular e WPF.
Repository	Abstrai o acesso a dados, isolando o sistema do banco de dados.
C4 Model	4 níveis de documentação: Contexto, Container, Componente, Código.
ATAM	Método de avaliação que analisa impactos de decisões nos atributos de qualidade.
SAAM	Foca em manutenibilidade e facilidade de modificação futura.

Trade-offs na Arquitetura

Não existe solução perfeita. Toda decisão traz vantagens e desvantagens que precisam ser equilibradas:

- Mais segurança pode comprometer performance
- Mais escalabilidade pode aumentar custos operacionais
- Consistência forte (SQL) vs. escalabilidade (NoSQL/MongoDB)

Aula 03 — Arquiteturas Distribuídas, Monolíticas, Microsserviços e Containers

Arquitetura distribuída: componentes executados em múltiplos nós (máquinas físicas ou virtuais) interconectados por rede.

Características da Arquitetura Distribuída

Conceito	Descrição
Transparência	Usuário não sabe em qual nó está a execução. Subtypes: localização, replicação, concorrência.

Heterogeneidade	Nos com diferentes OS, linguagens e hardware integrados via REST, gRPC, SOAP, CORBA.
Escalabilidade	Expansao horizontal (mais maquinas) ou vertical (mais recursos).
Tolerancia a falhas	Sistema continua funcionando mesmo com falha em componentes. Usa replicacao e failover.
Concorrencia	Processos interagem de forma sincrona ou assincrona via mensagens, filas ou APIs.

Desafios da Arquitetura Distribuida

- Latencia: operacoes em rede sao mais lentas que chamadas locais
- Consistencia de dados: complexidade do Teorema CAP (Consistencia, Disponibilidade, Tolerancia a Particoes)
- Seguranca: autenticao, autorizacao e criptografia obrigatorias
- Gerenciamento de estados: preferencia por arquiteturas stateless
- Monitoramento: necessidade de logs centralizados e rastreamento distribuido

Estrutura de Repositorio

Monorepo vs Multirepo

Conceito	Descrição
Monorepo	Todos os servicos no mesmo repositorio. Facilita compartilhamento de libs e CI/CD unificado.
Multirepo	Cada servico em repositorio separado em uma GitHub Organization. Maior autonomia por servico.

Aula 04 — Arquitetura MVC

MVC (Model-View-Controller): um dos padroes arquiteturais mais usados no desenvolvimento de software. Surgiu para separar dados, logica de negocio e interface grafica.

Componentes do MVC

Conceito	Descrição
Model	Dados e logica de negocio. Contem regras de validacao, persistencia e processamento. Nao depende de interface. Ex: classe Produto com calcularDesconto().
View	Camada de apresentacao. O que o usuario ve e interage. Nao contem regras de negocio. Ex: pagina HTML de lista de produtos.
Controller	Intermediario entre View e Model. Recebe requisicoes, processa e aciona o Model. Ex: metodo adicionarProdutoAoCarrinho().

Fluxo de Execucao no MVC

1. Usuario faz uma acao na View (ex: clicar em 'Adicionar ao carrinho')
2. Controller recebe a acao e interpreta a requisicao
3. Model e acessado para manipular dados ou regras de negocio
4. Resultado e enviado de volta para a View, que atualiza a interface

Beneficios e Desafios do MVC

MVC: Pros e Contras

Conceito	Descrição
Separacao de responsabilidades	Facilita manutencao e evolucao independente das camadas.
Reuso de codigo	Models podem ser utilizados em diferentes Views.
Escalabilidade	Equipes de front e back-end podem trabalhar em paralelo.
Testabilidade	Cada camada pode ser testada isoladamente.
God Object (risco)	Se mal implementado, Controller centraliza demais e vira um 'Deus' do sistema.
Complexidade desnecessaria	Pode ser exagerado para projetos muito pequenos ou simples.

Aula 05 — Arquitetura de Microservicos

Microservicos: estilo arquitetural que organiza uma aplicacao em modulos pequenos, independentes e fracamente acoplados, chamados de servicos. Cada servico trata de uma parte especifica do dominio.

Monolito vs Microservicos

Conceito	Descrição
Monolito	Unica aplicacao com todas as regras, dados e interfaces. Simples no inicio, dificil de manter em sistemas grandes.
Microservicos	Multiplos servicos independentes. Cada um pode usar tecnologias e bancos diferentes. Escalabilidade por servico especifico.

Caracteristicas Essenciais dos Microservicos

- Autonomia: desenvolvido, testado e implantado de forma independente
- Descentralizacao: cada servico pode ter seu proprio banco de dados
- Resiliencia: falha em um servico nao compromete o sistema
- Escalabilidade granular: aumenta capacidade apenas do servico critico
- Comunicacao clara: via APIs REST, gRPC ou mensageria

Comunicacao entre Servicos

Tipos de Comunicacao

Conceito	Descrição
Síncrona	Comunicacao direta via HTTP (REST) ou gRPC. Resultado imediato.
Assíncrona	Comunicacao via filas ou mensageria (Kafka, RabbitMQ). Desacoplado no tempo.
Orientada a eventos	Servicos reagem a eventos disparados por outros. Alta flexibilidade.

Padroes e Boas Praticas

Padroes de Microservices

Conceito	Descrição
API Gateway	Ponto central que organiza e roteia o acesso aos microservices.
Service Mesh	Camada de infraestrutura que gerencia comunicacao, seguranca e observabilidade.
Circuit Breaker	Evita falhas em cascata interrompendo chamadas a servicos com problema.
DDD (Bounded Context)	Cada microservico e modelado com base em um contexto de dominio.
Observabilidade	Logs, metricas e rastreamento distribuido com Prometheus, Grafana, OpenTelemetry.

Quando Usar Microservices

- Sistemas grandes e de dominio complexo
- Necessidade de escalabilidade individual por modulo
- Equipes grandes e distribuidas
- Alta frequencia de deploys independentes

ATENCAO: Para sistemas pequenos ou equipes reduzidas, arquitetura monolitica ainda e mais indicada.

Tecnologias Relacionadas

Stack de Microservices

Conceito	Descrição
Back-end	Node.js + Express para implementacao de cada servico.
Front-end	React, Vue ou Angular para consumo das APIs dos servicos.
Infraestrutura	Docker (containers) + Kubernetes (orquestracao).
Monitoramento	Prometheus, Grafana, ELK Stack, OpenTelemetry.

PARTE 2 — SIMULADO COM GABARITO (20 Questoes)

Todas as questoes sao de multipla escolha com 5 alternativas. A resposta correta e indicada com explicacao aplicada ao conteudo das aulas.

Questão 1

Qual das alternativas abaixo melhor descreve o conceito de Arquitetura de Software?

- A) E o codigo fonte final de uma aplicacao, incluindo testes e scripts.
- B) E apenas o diagrama visual gerado por ferramentas CASE para documentacao.
- C) E a estrutura fundamental de um sistema, composta por componentes, suas relacoes e os principios que guiam seu projeto e evolucao.
- D) E exclusivamente a escolha do banco de dados e da linguagem de programacao.
- E) E o conjunto de metodologias ageis adotadas pela equipe de desenvolvimento.

Resposta Correta: C

Explicação: Arquitetura responde 'como o sistema sera organizado'. Define blocos principais, como se comunicam e orienta decisoes tecnicas. Nao e so codigo, nem so documentacao — e a estrutura que sustenta tudo.

Questão 2

Qual e a principal diferenca entre Escalabilidade Vertical e Escalabilidade Horizontal?

- A) Vertical aumenta instancias; horizontal melhora o hardware.
- B) Vertical melhora recursos da mesma maquina (CPU/memoria); horizontal adiciona novas instancias do sistema.
- C) Sao sinonimos: ambas descrevem o mesmo processo de crescimento do sistema.
- D) Horizontal e usada apenas em bancos NoSQL; vertical em bancos SQL.
- E) Vertical utiliza Kubernetes; horizontal utiliza Docker Swarm obrigatoriamente.

Resposta Correta: B

Explicação: Escalabilidade vertical = trocar o motor do carro por um mais potente. Horizontal = adicionar mais carros. Em sistemas reais: vertical tem limite fisico de hardware; horizontal (Docker + Kubernetes) e a abordagem preferida para sistemas cloud-native.

Questão 3

No padrao MVC, qual componente e responsavel por receber as requisicoes do usuario e coordenar as interacoes entre Model e View?

- A) Model
- B) View
- C) Repository
- D) Controller
- E) Service Layer

Resposta Correta: D

Explicação: O Controller e o intermediário: recebe a ação do usuário na View (ex: clicar 'Comprar'), aciona o Model para processar dados e devolve o resultado para a View atualizar a interface. Sem Controller bem definido, o sistema vira um 'God Object'.

Questão 4

Qual das seguintes características NÃO é uma vantagem da Arquitetura de Microserviços?

- A) Escalabilidade granular por serviço individual.
- B) Resiliência — falha em um serviço não derruba o sistema inteiro.
- C) Simplicidade operacional, pois requer pouca infraestrutura e monitoramento.
- D) Autonomia de desenvolvimento e deploy por serviço.
- E) Possibilidade de cada serviço usar tecnologias e bancos diferentes.

Resposta Correta: C

Explicação: Microserviços exigem MAIOR complexidade operacional: orquestração com Kubernetes, pipelines CI/CD, monitoramento distribuído, service mesh. A simplicidade operacional é uma característica do monolito, não de microserviços.

Questão 5

Em qual situação a Arquitetura Monolítica é MAIS recomendada que a de Microserviços?

- A) Quando o sistema precisa de alta escalabilidade e muitos deploys independentes.
- B) Quando equipes grandes e distribuídas trabalham em domínios separados.
- C) Quando o sistema é pequeno, o domínio é simples e a equipe é reduzida.
- D) Quando o sistema processa milhões de transações por segundo.
- E) Quando a empresa precisa de tolerância a falhas máxima.

Resposta Correta: C

Explicação: A própria aula afirma: 'Para sistemas pequenos ou equipes reduzidas, arquitetura monolítica ainda é mais indicada.' Microserviços trazem complexidade operacional que só se justifica quando o sistema é grande e o contexto exige escala.

Questão 6

O Teorema CAP, relevante em arquiteturas distribuídas, afirma que um sistema distribuído pode garantir simultaneamente no máximo dois dos três atributos. Quais são eles?

- A) Coesão, Acoplamento e Performance.
- B) Consistência, Disponibilidade e Tolerância a Partições.
- C) Completude, Atomicidade e Persistência.
- D) Controle, Autenticação e Privacidade.
- E) Cobertura, Abrangência e Proteção.

Resposta Correta: B

Explicação: CAP = Consistency (C), Availability (A), Partition Tolerance (P). Um sistema distribuído não consegue garantir os 3 ao mesmo tempo. Ex: bancos NoSQL como MongoDB priorizam A e P sacrificando C; bancos SQL priorizam C e A sacrificando P.

Questão 7

Qual padrão arquitetural organiza serviços independentes expostos por interfaces bem definidas e é considerado o precursor dos microsserviços?

- A) MVC (Model-View-Controller)
- B) Pipeline/Batch
- C) Event-Driven Architecture
- D) SOA (Service-Oriented Architecture)
- E) Arquitetura em Camadas

Resposta Correta: D

Explicação: SOA (Arquitetura Orientada a Serviços) organizou sistemas em serviços reutilizáveis com interfaces padronizadas. Os microsserviços evoluíram desse conceito, tornando os serviços ainda menores, mais autônomos e com comunicação mais leve (REST/gRPC).

Questão 8

No contexto do Modelo C4, qual nível descreve como o sistema interage com atores externos (usuários, sistemas externos)?

- A) Nível Código
- B) Nível Componente
- C) Nível Container
- D) Nível Contexto
- E) Nível Implantação

Resposta Correta: D

Explicação: O Modelo C4 tem 4 níveis: Contexto (visão geral com atores externos), Container (subsistemas como API, banco, front-end), Componente (módulos internos) e Código (classes e funções). Para prova e entrevista, lembrar: Contexto = visão mais externa e abstrata.

Questão 9

O padrão API Gateway em microsserviços tem qual função principal?

- A) Substituir o banco de dados dos microsserviços.
- B) Ser o ponto central que organiza e roteia o acesso aos diferentes microsserviços.
- C) Monitorar logs e métricas de cada serviço individualmente.
- D) Gerenciar o versionamento de código dos serviços.
- E) Criar um banco de dados compartilhado entre todos os serviços.

Resposta Correta: B

Explicação: O API Gateway é a 'porta de entrada' do sistema: o cliente faz uma única chamada ao Gateway, que roteia internamente para o serviço correto. Também pode centralizar autenticação, rate limiting e logging. Sem ele, o cliente precisaria conhecer o endereço de cada microserviço.

Questão 10

Qual técnica de avaliação arquitetural analisa os impactos das decisões arquiteturais nos atributos de qualidade antes da implementação?

- A) TDD (Test-Driven Development)
- B) SCRUM
- C) ATAM (Architecture Tradeoff Analysis Method)
- D) Clean Architecture
- E) SOLID

Resposta Correta: C

Explicação: ATAM permite avaliar a arquitetura ANTES de construir o sistema, identificando riscos, trade-offs e pontos fracos. SAAM complementa com foco em manutenibilidade. São as técnicas formais de avaliação arquitetural cobertas na disciplina.

Questão 11

Em React, seguindo conceitos análogos ao MVC, o que representa o 'Model' da aplicação?

- A) Os arquivos CSS que estilizam os componentes.
- B) As rotas definidas com React Router.
- C) O estado da aplicação gerenciado com useState ou Redux.
- D) Os componentes JSX que renderizam a interface.
- E) As chamadas de API feitas com fetch ou axios.

Resposta Correta: C

Explicação: No contexto React/MVC: Model = estado (useState, Redux, Context API); View = componentes JSX que exibem dados; Controller = funções que atualizam o estado e tratam interações. O dado em si — o estado — é o Model.

Questão 12

Qual conceito descreve as consequências negativas acumuladas de decisões arquiteturais ruins ou apressadas, como acoplamento excessivo e ausência de testes?

- A) Refatoração Contínua
- B) Dívida Técnica
- C) Redundância Operacional
- D) Sobrecarga de Serviço
- E) Deficit de Modularidade

Resposta Correta: B

Explicação: Dívida Técnica e quando você toma atalhos que funcionam no curto prazo mas cobram juros no futuro: código sem testes, acoplamento alto, falta de documentação. A solução típica é refatoração incremental — no caso extremo, migração de monolito para microsserviços.

Questão 13

Na Arquitetura Distribuída, o que significa 'Transparência de Localização'?

- A) O código-fonte é público e acessível a todos os usuários.
- B) O usuário pode acessar serviços sem saber em qual nó físico estão sendo executados.
- C) A aplicação pode ser usada em diferentes países sem restrição legal.
- D) O banco de dados fica visível para todos os serviços sem autenticação.
- E) Os logs do sistema são acessíveis por qualquer desenvolvedor da equipe.

Resposta Correta: B

Explicação: Transparência de Localização é um dos pilares da arquitetura distribuída: o usuário interage normalmente sem saber se o serviço está no servidor A, B ou C. Isso é fundamental para balanceamento de carga, failover e escalabilidade transparente.

Questão 14

Qual é a diferença fundamental entre comunicação Síncrona e Assíncrona em microsserviços?

- A) Síncrona usa HTTP; assíncrona usa HTTPS.
- B) Síncrona exige banco de dados compartilhado; assíncrona não.
- C) Síncrona aguarda resposta imediata (HTTP/gRPC); assíncrona usa filas/mensageria como Kafka sem esperar resposta imediata.
- D) Síncrona é mais segura; assíncrona é mais rápida porém insegura.
- E) Não existe diferença prática: são implementações diferentes da mesma ideia.

Resposta Correta: C

Explicação: Na comunicação síncrona o serviço A chama o serviço B e espera a resposta (bloqueante). Na assíncrona, A publica uma mensagem em uma fila (Kafka, RabbitMQ) e continua executando — B processa quando estiver disponível. Assíncrona aumenta resiliência e desacoplamento.

Questão 15

O Modelo 4+1 de Kruchten define visões arquiteturais para descrever um sistema. Quais são as 5 visões?

- A) Frontal, Lateral, Superior, Inferior e Diagonal.
- B) Lógica, Processo, Desenvolvimento, Física e Casos de Uso.
- C) Dados, Serviços, Interface, Segurança e Infraestrutura.
- D) Negócio, Tecnologia, Aplicação, Dados e Segurança.
- E) Estática, Dinâmica, Funcional, Não-Funcional e Temporal.

Resposta Correta: B

Explicação: O modelo 4+1 de Kruchten: Logica (o que o sistema faz), Processo (como processos interagem), Desenvolvimento (organizacao para devs), Fisica (implantacao em hardware) e Casos de Uso (cenarios de validacao — o '+1'). E base para muitas questoes de certificacao.

Questão 16

Em uma arquitetura de microservicos para e-commerce, qual padrao evita que a falha do servico de Pagamentos derrube tambem o servico de Pedidos?

- A) API Gateway
- B) Service Mesh
- C) Circuit Breaker
- D) CQRS
- E) Saga Pattern

Resposta Correta: C

Explicação: Circuit Breaker (Disjuntor): quando um servico detecta falhas repetidas em outro, 'abre o disjuntor' e para de fazer chamadas por um periodo. Isso evita falhas em cascata. Implementado com bibliotecas como Resilience4j (Java) ou com Istio no Service Mesh.

Questão 17

No padrao MVC bem implementado, onde devem ficar as regras de negocio (ex: calcular desconto, validar CPF)?

- A) Na View, para que o usuario veja o resultado imediatamente.
- B) No Controller, pois ele coordena toda a aplicacao.
- C) No banco de dados, em procedures armazenadas.
- D) No Model, que representa dados e logica de negocio.
- E) No arquivo de rotas, para centralizar toda logica de negocio.

Resposta Correta: D

Explicação: Model = dados + logica de negocio. Regras como calcularDesconto(), validarCPF() ou verificarEstoque() pertencem ao Model. Colocar logica no Controller gera o anti-pattern 'Fat Controller / God Object', que e um dos principais desafios do MVC mal implementado.

Questão 18

Qual das seguintes afirmacoes sobre a relacao entre Engenharia de Software e Arquitetura de Software e CORRETA?

- A) Arquitetura de Software e mais ampla que Engenharia de Software.
- B) Sao conceitos completamente separados e independentes.
- C) Arquitetura de Software e um subconjunto da Engenharia de Software, com foco na estrutura e nos requisitos de qualidade.
- D) Engenharia de Software trata apenas de codificacao; Arquitetura trata de gestao de projetos.

E) Ambas tratam exclusivamente de metodologias ageis como Scrum e Kanban.

Resposta Correta: C

Explicação: Engenharia de Software abrange TODO o ciclo de vida: requisitos, analise, projeto, codificacao, testes, manutencao e gestao. Arquitetura e um subconjunto especializado, focado em como estruturar o sistema para atender requisitos de qualidade (escalabilidade, segurança, etc).

Questão 19

Qual abordagem de repositório em microserviços recomenda que TODOS os serviços fiquem no mesmo repositório compartilhado?

- A) Multirepo
- B) GitHub Organization
- C) Monorepo
- D) Distributed Repository
- E) Federated Codebase

Resposta Correta: C

Explicação: Monorepo = todos os serviços no mesmo repositório. Vantagens: fácil compartilhar libs comuns (autenticacao, mensageria), CI/CD unificado e visibilidade do sistema inteiro. Desvantagem: pode crescer muito e dificultar autonomia de equipes. Multirepo dá mais autonomia, mas exige mais governanca.

Questão 20

Em uma arquitetura em camadas classica com 3 camadas, qual e a ordem correta de cima para baixo?

- A) Dados > Logica de Negocio > Interface do Usuario
- B) Interface do Usuario > Logica de Negocio > Dados
- C) Logica de Negocio > Interface do Usuario > Dados
- D) Dados > Interface do Usuario > Logica de Negocio
- E) Interface do Usuario > Dados > Logica de Negocio

Resposta Correta: B

Explicação: A arquitetura em 3 camadas (three-tier) vai de UI > Logica > Dados: Interface do Usuario (apresentacao) comunica com a Logica de Negocio (regras/processamento), que por sua vez acessa a camada de Dados (banco). Cada camada so se comunica com a adjacente — isso garante separacao de responsabilidades.

FIM DO SIMULADO

PARTE 3 — ATIVIDADES OFICIAIS DA DISCIPLINA

Questões extraídas das atividades avaliativas do Prof. Esp. Leonardo Dias. Formato original com gabarito comentado.

Atividade 1 — Conceitos Fundamentais de Arquitetura de Software

Questão 21

Qual das alternativas melhor define arquitetura de software?

- A) O código fonte detalhado do sistema.
- B) Um conjunto de práticas de programação orientada a objetos.
- C) A estrutura fundamental de um sistema, seus componentes e a forma como interagem.
- D) Apenas o design visual de interfaces do usuário.

Resposta Correta: C

Explicação: A arquitetura de software não é código nem design visual — é a estrutura de alto nível que define blocos, relações e princípios de evolução do sistema. Pense na planta de uma casa: define a estrutura antes da construção começar.

Questão 22

Entre os atributos de qualidade arquitetural, qual está relacionado à capacidade de o sistema crescer em número de usuários sem perder desempenho?

- A) Modularidade
- B) Escalabilidade
- C) Manutenibilidade
- D) Portabilidade

Resposta Correta: B

Explicação: Escalabilidade = crescer sem perder desempenho. Pode ser vertical (mais recursos na mesma máquina) ou horizontal (adicionar novas instâncias via Docker/Kubernetes). Não confundir com Modularidade (organização) ou Manutenibilidade (facilidade de alterar).

Questão 23

Qual analogia foi usada para explicar o papel da arquitetura de software?

- A) Receita de bolo.
- B) Planta de uma casa antes da construção.
- C) Manual de instruções de eletrodomésticos.
- D) Montagem de um quebra-cabeça.

Resposta Correta: B

Explicação: A analogia classica das aulas: assim como a planta de uma casa define divisao de comodoss, localizacao de portas e estrutura eletrica antes da obra, a arquitetura de software define a estrutura do sistema antes de comecar a codar. Sem planta, tudo fica improvisado e caro de corrigir.

Questão 24

Qual atributo arquitetural facilita a correcao de erros e a evolucao do sistema sem comprometer outras partes?

- A) Escalabilidade
- B) Modularidade
- C) Manutenibilidade
- D) Performance

Resposta Correta: C

Explicação: Manutenibilidade = facilidade de modificar o sistema com seguranca e rapidez. Depende de boas praticas como Clean Code, testes automatizados e separacao clara de responsabilidades. Analogia das aulas: trocar a fiacao da sala sem derrubar a parede errada.

Questão 25

Na comparacao entre arquitetura, design e implementacao, qual e a caracteristica da implementacao?

- A) Definir blocos principais e suas interacoes.
- B) Escolher padroes de design como MVC ou Repository.
- C) Escrever codigo, testes e scripts.
- D) Decidir estilos arquiteturais e tecnologias.

Resposta Correta: C

Explicação: Os tres niveis: Arquitetura (alto nivel — blocos e interacoes), Design (nivel intermediario — padroes e organizacao interna) e Implementacao (nivel mais baixo — codigo fonte, testes, scripts). A implementacao e o 'tijolo por tijolo' da construcao.

Atividade 2 — Arquiteturas Distribuidas e Estilos Arquiteturais

Questão 26

Associe o modelo Cliente-Servidor a sua caracteristica principal:

- A) Camadas de apresentacao, logica e dados.
- B) Cliente solicita servicos e o servidor responde.
- C) Servicos independentes que se comunicam por APIs.
- D) Publicacao e consumo de eventos em filas.

Resposta Correta: B

Explicação: Cliente-Servidor e o modelo mais básico de distribuição: clientes (browsers, apps) fazem requisições e servidores processam e respondem. A opção A descreve arquitetura em 3 camadas; C descreve microsserviços; D descreve Event-Driven.

Questão 27

Na arquitetura em três camadas, quais são as camadas principais?

- A) Interface, API e banco de dados.
- B) Lógica de negócio, apresentação e dados.
- C) Cliente, servidor e cache.
- D) Front-end, back-end e mensageria.

Resposta Correta: B

Explicação: As 3 camadas clássicas são: Apresentação (UI — o que o usuário vê), Lógica de Negócio (regras, processamento) e Dados (banco de dados). Cada camada só se comunica com a adjacente, garantindo separação de responsabilidades e manutenibilidade.

Questão 28

Na arquitetura de microsserviços, cada serviço:

- A) É grande, centralizado e compartilha o mesmo banco.
- B) É pequeno, independente e comunica-se por protocolos leves.
- C) Não pode ser replicado.
- D) Funciona apenas com SOAP.

Resposta Correta: B

Explicação: Microsserviços são pequenos, autônomos e fracamente acoplados. Cada um pode ter seu próprio banco de dados, ser desenvolvido em tecnologias diferentes e comunicar via REST, gRPC ou mensageria. O compartilhamento de banco entre serviços é justamente o anti-pattern a evitar.

Questão 29

A arquitetura orientada a eventos é caracterizada por:

- A) Interações síncronas entre cliente e servidor.
- B) Processamento em lote de informações.
- C) Comunicação baseada em mensagens assíncronas em filas/tópicos.
- D) Uso exclusivo de banco de dados relacionais.

Resposta Correta: C

Explicação: Event-Driven = componentes publicam eventos e outros consomem de forma assíncrona. Ex: ao finalizar uma compra, um evento é publicado no Kafka; o serviço de notificações consome e envia o e-mail sem que o serviço de pedidos precise esperar. Desacoplamento e resiliência são os grandes benefícios.

Questão 30

Qual é um dos principais desafios das arquiteturas distribuídas?

- A) Modularidade.
- B) Consistência de dados e latência.
- C) Separação de responsabilidades.
- D) Escalabilidade horizontal.

Resposta Correta: B

Explicação: Em sistemas distribuídos, manter consistência entre réplicas é complexo (Teorema CAP). Operações em rede são mais lentas e sujeitas a falhas que chamadas locais. Modularidade e separação de responsabilidades são benefícios, não desafios. Escalabilidade horizontal é uma vantagem, não problema.

Atividade 3 — Situação-Problema em Arquitetura de Software

Cenário: uma empresa de e-commerce com sistema monolítico enfrenta lentidão com o aumento de usuários e precisa migrar para uma arquitetura que permita escalar serviços de forma independente e aumentar a resiliência.

Questão 31

Qual estilo arquitetural deve ser adotado para resolver esse cenário?

- A) Arquitetura em três camadas.
- B) Arquitetura orientada a microsserviços.
- C) Arquitetura cliente-servidor.
- D) Arquitetura monolítica otimizada.

Resposta Correta: B

Explicação: O cenário clássico de migração: monólito com dificuldade de escalar > microsserviços. A arquitetura de microsserviços permite escalar apenas os serviços críticos (ex: catálogo de produtos no pico de Black Friday) sem afetar os demais. As outras opções não resolvem o problema de escalabilidade granular.

Questão 32

Um dos principais desafios dessa nova arquitetura será:

- A) Modularidade insuficiente.
- B) Consistência de dados entre serviços.
- C) Falta de escalabilidade horizontal.
- D) Dificuldade de separar camadas.

Resposta Correta: B

Explicação: Ao separar em microsserviços, cada um com seu banco próprio, não há mais transações globais. Garantir consistência eventual entre serviços (ex: pedido criado mas pagamento ainda não

confirmado) e um dos maiores desafios operacionais. Padrão Saga é usado para gerenciar transações distribuídas.

Questão 33

Para lidar com a comunicação entre os serviços de forma assíncrona e desacoplada, a equipe pode utilizar:

- A) Padrão MVC.
- B) Balanceador de carga.
- C) Fila de mensageria como RabbitMQ ou Kafka.
- D) API SOAP centralizada.

Resposta Correta: C

Explicação: Para comunicação assíncrona e desacoplada entre microsserviços, filas de mensageria são a solução: RabbitMQ (filas simples, mensagens ponto-a-ponto) ou Kafka (streaming de eventos de alto volume, retenção e replay). Balanceador de carga resolve distribuição de requisições síncronas, não desacoplamento.

Questão 34

Qual atributo de qualidade arquitetural será mais diretamente beneficiado pela migração para microsserviços?

- A) Escalabilidade.
- B) Portabilidade.
- C) Usabilidade.
- D) Modularidade.

Resposta Correta: A

Explicação: O problema era escalar o monólito inteiro para atender a demanda. Com microsserviços, a empresa pode escalar apenas o serviço de Catálogo de Produtos durante o pico, sem subir novas instâncias de Pagamentos ou Notificações. Escalabilidade granular é o principal ganho direto nesse cenário.

Questão 35

Se a equipe optar por armazenar cada serviço em um repositório separado no GitHub (multirepo), uma boa prática será:

- A) Centralizar todo o código em um único projeto.
- B) Criar uma organização no GitHub e separar cada serviço em um repositório próprio.
- C) Unificar os pipelines de CI/CD em apenas um repositório global.
- D) Evitar documentar cada serviço para não gerar redundância.

Resposta Correta: B

Explicação: No modelo multirepo: criar uma GitHub Organization (ex: org-ecommerce) e dentro dela um repositório por serviço (pedidos-service, pagamentos-service, etc.). Cada repositório tem seu próprio pipeline CI/CD e documentação mínima. Centralizar tudo em um projeto único seria o modelo Monorepo, não multirepo.

FIM DAS ATIVIDADES OFICIAIS

Total: 35 questões com gabarito comentado | Prof. Esp. Leonardo Dias — UniFOA